Due: Friday February 17th, 5pm (week 5)
This assignment is worth 25% of your overall marks.

PLEASE READ THESE INSTRUCTIONS CAREFULLY.

Submission instructions.
Before submitting, ensure ALL of your python files are located in a folder. The folder should labelled in the following format:

StudentID_assignment_1

For example, if your student ID number is *123456*, then your folder should be named:

123456_assignment_1

The files that should be embedded in this folder are:

- main.py (containing main code, with comments)
- bubble.py (containing `Bubble` class, with comments)
- paddle.py (containing `Paddle` class, with comments)
- bullet.py (containing `Bullet` class, with comments)

Zip (compress) the folder before submitting.

You should complete these actions by the due date specified above. No late work will be accepted. The due date is final!

Plagiarism check
The Python code of the practical will be checked for plagiarism following the UCC guidelines: https://www.ucc.ie/en/media/support/recordsandexaminations/documents/UCCPlagiarismPolicy-November2017V1.0-CLEAN.pdf
Plagiarism detection in a computer program is a simple process and experienced programmers can easily see if two programs are "modified" versions of the same one.

Project: Bubble Shooter!

You will be creating a Bubble Shooter Game using pygame and circuitpython! The objective of the game is to create a paddle to shoot a bullet toward a bubble. The bullet should hit the bubble before that bubble passes a line drawn on the screen. If the bullet fails to hit the bubble in that time, the bubble relocates to a position on the screen before the line (this position can be random or of your choice). In the final project, the paddle should move in response to gesture movements and the bullet should be controlled by a button press. Note that the following characteristics must be implemented:

- The paddle should only be able to move horizontally OR vertically. It should not be able to move in both directions.
- The paddle should not be able to move outside the bounds of the window/screen.
- The bullet should move with the paddle when it is not triggered
- When triggered, the bullet should appear to be "shooting" from the paddle
- Only one bullet should appear on the screen at a time

- The bubble should be able to "bounce" against the opposite walls. It should relocate when it moves beyond the position of the line on the screen (i.e. the pygame display window) either horizontally or vertically.

This Assignment - Lab Assessment 1: Building the main pygame elements of the game

You will be creating classes (`Paddle` , `Bubble` and `Bullet`) and creating interactions between these classes in the main code.

Explanatory video:
https://ucc.cloud.panopto.eu/Panopto/Pages/Viewer.aspx?id=f3a2dd1f-8131-4457-9478-acc2016f87de

Detailed guidelines

Note, you will be creating separate classes to control the gameplay, in the following files:

- In paddle.py, create the class `Paddle`, which is the blueprint for the paddle
- In bubble.py, create the class `Bubble`, which is the blueprint for the bubble
- In bullet.py, create the class Bullet, which is the blueprint for the bullet
- In main.py, create the main interactions of the game and create appropriate instances of the `Paddle`, `Bullet` and `Bubble` classes. This includes:
    - Creating a display window with a size of your choice.
    - Drawing a line, which marks the area where the bubble should not pass before being hit by the bullet
    - The bubble should be able to move diagonally and should automatically move. Appropriate methods should be called so that the bubble collides with the "walls" of the screen, and relocate once it is in contact with the line.
    - The paddle should be able to move either:
      a) left and right, *with the left and right arrow keys on the keyboard respectively*; or
      b) Up and down, with the up and down arrow keys on the keyboard respectively.
      The paddle should not move off the screen. The paddle should continue moving when the user has the appropriate key pressed, and stop moving once released.
    - A bullet should appear to be moving on top of the paddle
    - When a user presses a key of your choice (other than the keys already in use for the application), the bullet should appear to "shoot" from the paddle, and move horizontally <u>or</u> vertically, depending on the location of the paddle. Note that:
        - No more than one bullet should appear at a time. The paddle cannot launch another bullet until the original bullet launched has disappeared
        - For assignment 1, there is no need to create interactions between the bullet and the bubble yet. This means that for now, the bubble does not need to relocate if the bullet collides with it.
    - The game should continue to play and the display window should remain open, until the user closes the window by clicking the "x" on the top corner of the screen.

Creating classes

Note that all classes below make use of pygame, so you should make sure it is imported in the file you are creating this class in.

1	**Creating the Paddle class**
A	The `Paddle` class contains the following variables, **<u>all of which should be private</u>**. You may decide whether the variables below are class or instance variables. However, you will need to defend your choice in your comments. (i) Variables for the coordinates of the paddle (including x, y, width, height) (ii) Variables for the paddle colour and speed

B	`__init__(self, x, y, color, speed):`
	The constructor should be used to set up the `Paddle` class. This might include the coordinates, colour and speed of the Paddle (as defined above). You will need to defend your choice on parameters in comments.
C	**Create a string representation:** when the object is printed to the screen, it should contain reader-friendly information concerning the object's instance. This should include the colour and coordinates of the Paddle.
D	`draw(self, display):`
	This method draw the paddle on the display window. To draw the paddle, you will need to make use of pygame features (e.g. rectangles, circles, lines). **Note you should not need to draw the paddle in other methods.**
E	`move_left(self, moveleft_trigger):` **OR** `move_up(self, moveup_trigger):`
	This method should move the paddle left or up, depending on which method you choose to implement. The speed the paddle moves should be defined in an instance variable, containing a value of your choice. Note the following constraints: • The paddle should not be able to move off the left of the screen OR the top of the screen (depending on which method has been implemented). Note that you should not draw the paddle in this method.
F	`move_right(self, display_width, moveright_trigger):` **OR** `move_down(self, display_height, movedown_trigger):`
	This method should move the paddle right or down, depending on which method you choose to implement. The speed the paddle moves should be defined in an instance variable, containing a value of your choice. Note the following constraints: • The paddle should not be able to move off the right of the screen OR the bottom of the screen (depending on which method has been implemented). Note that you should not draw the paddle in this method.
G	**Getters for the coordinates (x, y) and size of the paddle**
	Create these getter methods.

2	**Creating a Bubble class**
A	The `Bubble` class contains the following variables, **all of which should be private**. You may decide whether the variables below are class or instance variables. However, you will need to defend your choice in your comments. (i) Variables for the coordinates of the bubble (including x, y, radius or size in width/height) (ii) Variables for the bubble speed on the x and y axis
B	`__init__(self, x, y, colour, radius, speed_x, speed_y):`
	The constructor should be used to set up the Bubble class. This might include the coordinates, colour, speed of the Bubble as shown above. You will need to defend your choice on parameters in comments.
C	**Create a string representation:** You should ensure that when the object is printed to the screen, it contains reader-friendly information concerning the object's instance, including the color and position of the Bubble.
D	`draw(self, display):`
	This method draw the bubble on the display window. The display window is passed in with this method. To draw the bubble, you will need to make use of pygame features (e.g. rectangle, circle or line). **Note you should not need to draw the bubble in other methods.**
E	`move(self, horizontal_bound, vertical_bound):`

	This method should move the bubble diagonally. The speed the bubble moves should be defined in instance variables, containing a value of your choice. Note the following constraints for the bubble: • The bubble's movement should be constrained within the bounds defined in the parameters `horizontal_bound`, `vertical_bound`. • The bubble should bounce (i.e. change direction) once it collides with the `horizontal_bound`, • The bubble should relocate once it collides with the `vertical_bound`. The relocation could be random; however, it should not be hardcoded and should take into account the `horizontal_bound`, `vertical_bound`.
F	**`Getters for the x, y, and size the bubble`**
	Create getters to retrieve (i.e. return) the x, y, radius (or size) of the bubble.

3	**`Creating a Bullet class`**
A	The `Bullet` class contains the following variables, **<u>all of which should be private</u>**. You may decide whether the variables below are class or instance variables. However, you will need to defend your choice in your comments. (i) Variables for the coordinates of the bullet (including x, y, radius or size). Note that the bullet should not be initially visible on the screen (ii) Variables for the bullet colour (iii) Variables for the bullet speed (iv) A Boolean variable for determining whether the bullet is has been triggered or not, initially set to `False`
B	**`__init__(self, width, height, colour, speed):`**
	The constructor should be used to set up the Bullet class. This might include the size, colour, speed of the Bullet as shown above. However, your implementation may be different depending on what shape you have decided to draw (e.g. a line, rectangle or circle)
C	**Create a string representation:** when the object is printed to the screen, it should contain reader-friendly information concerning the object's instance. This should include the colour and position of the Bullet.
D	**`draw(self, display):`**
	This method draw the bullet on the display window. The display window is passed in with this method. To draw the bullet, you will need to make use of pygame features. **<u>Note you should not need to draw the bullet in other methods.</u>**
E	**`move(self, pos: list, dir)`**

	This method accepts the following parameters:
	• pos (list): the position where the bullet should be located, if not triggered. It is in [x,y] format, and you may assume it will always be in that format.
	• dir (str): The direction in which the bullet should move. There are four possible directions: "left", "right", "up" or "down".
	
	This method should move the bullet accordingly, depending on whether it has been triggered or not (which is defined in the instance variable created to trigger the bullet in A(iv)):
	• If the bullet has not been triggered, it should move with the coordinates defined within pos.
	• If the bullet has been triggered, it should appear to "emerge" from the paddle. Therefore, it should:
	  • move horizontally, in the direction the bubble is located, if the paddle is located on the left/right hand side of the screen.
	  • move vertically, in the direction the bubble is located, if the paddle is located on the top/bottom of the screen.

F	**`set_trigger(self, triggered: bool)`**

	This method has been designed to be called when the user triggers the bullet to emerge from the paddle. Other than `self`, this method passes in the following parameters: • `triggered (bool)`: a boolean determining whether the bullet has been triggered This method should set the instance variable for triggering the bullet, as defined in A(iv) to the value inside `triggered`.
I	**`wall_collision(self):`**
	If the bullet collides with the "wall" behind the bubble, it means that the bullet missed the bubble. This method should check if the bullet has collided with the wall behind the bubble. Depending on where your bubble is located, you may need to pass in an additional parameter to determine the wall location. • If the bullet has collided with the wall behind the bubble, the variable to control the bullet moving should be set to `False`. • If the bullet has not collided with the wall behind the bubble, the variable controlling the bullet moving should be set to `True`.
J	**Getters for the coordinates and size (or radius) of bullet**
	Create getters to retrieve (i.e. return) the x, y, radius (or size) of the bullet

		Not achieved	Somewhat achieved	Nearly there	Achieved
Definition of game classes	~~10~~ 15	Pygame is not used or referenced. The methods are defined, but most of them are not completed or working properly. **Comments** are not included or comments do not describe what each method in the class does.	Pygame is imported and used appropriately to draw shapes or images for the bullet, paddle and bubble. Most of the methods are working, but there are some issues with the methods causing them not to work as described. However the student highlights those issues as comments and provides a possible reason why the code is not working. **Comments:** The student explains what the methods are doing in their own words, demonstrating clear understanding. The student also explains in the comments how each class is designed both draw and manipulate the objects as separate GUI components using their own words.	Pygame is imported and used appropriately to draw shapes or images for the bullet, paddle and bubble. The methods inside the Bullet, Paddle and Bubble class are defined and working correctly. There may be 1-2 issues, but these are small and do not interfere with the code working. Where there are issues, the student clearly provides a comment as to what they are and how the student will address these in the next iteration. **Comments:** The student explains what the methods are doing in their own words, demonstrating clear understanding. The student also explains in the comments how each class is designed to draw and manipulate the objects as separate GUI components using their own words.	Pygame is imported and used appropriately to draw shapes or images for the bullet, paddle and bubble. The methods inside the Bullet, Paddle and Bubble class are defined and working correctly. There are no issues with the code and everything works as described. **Comments:** The student explains what the methods are doing in their own words, demonstrating clear understanding. The student also explains in the comments how each class is designed both draw and manipulate the objects as separate GUI components using their own words.
Gameplay interactions	10	No, or incorrect use of objects in the main code. The Bullet, Paddle and Bubble classes are not interacting in the main.py code. None of the pygame elements are applied, or are mostly not working. There is no pygame window displayed. There is no game loop. No comments in the code.	The Bullet, Paddle and Bubble classes are imported in main.py. Instances of the class are created. Some of the pygame elements are applied, fundamentally the window. The objects (paddle/bullet/bubble/line) are drawn to the screen. Objects are moving but not properly. Event driven programming is applied to control object movements and collisions, where necessary. **Comments:** the student explains, in their own words, what the pygame interactions are doing. Additionally, the student explains event driven programming, and how it was employed to control each of the objects created in their own words. In cases where interactions are not working, the student explains how event driven programming could have been employed in their own words.	The Bullet, Paddle and Bubble classes are imported in main.py. Instances of the class are created. Most of the pygame elements are applied, fundamentally the window. The objects (paddle/bullet/bubble/line)are drawn to the screen. Event driven programming is applied to control object movements and collisions, where necessary. **Comments:** the student explains, in their own words, what the pygame interactions are doing. Additionally, the student explains event driven programming, and how it was employed to control each of the objects created in their own words. In cases where interactions are not working, the student explains how event driven programming could have been employed in their own words.	The Bullet, Paddle and Bubble classes are imported in main.py. Instances of the class are created. All pygame elements described are working. The objects (paddle/bullet/bubble/line)are drawn to the screen. Objects are moving and working as described. Event driven programming is applied to control object movements and collisions, where necessary. **Comments:** the student explains, in their own words, what the pygame interactions are doing. Additionally, the student explains event driven programming, and how it was employed to control each of the objects created in their own words. In cases where interactions are not working, the student explains how event driven programming could have been employed in their own words.